UNIVERSIDAD PERUANA LOS ANDES
FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

MODELO FÍSICO DE LA BASE DE DATOS COMPUTECH

ESTRUCTURA, VISTAS, PROCEDIMIENTOS, CONSULTAS Y EXPORTACIÓN INTEGRAL

ACTIVIDAD 01 | SEMANA 06

ALUMNO:

➢ **QUISPE TICA RUTH NAYELI**

CURSO:

➢ **DESARROLLO DE APPS**

CICLO:

➢ **IV**

SECCIÓN:

➢ **A1**

HUANCAYO - PERÚ
2026

Presentación

CompuTechDB es la base de datos en SQL Server que guarda toda la información de la aplicación web *CompuTech – Sistema de Gestión de Productos* (proyecto PaginaWeb-Proyecto, Java con Servlets, JSP y Tomcat 11). Tiene **8 tablas** que cubren el catálogo de productos, las cuentas de usuario y el proceso de compra.

La aplicación se conecta con JDBC (driver `mssql-jdbc`) usando los datos de `db.properties`: servidor `localhost:1433`, base `CompuTechDB`. Todo el acceso a datos pasa por las clases DAO (`ProductoDAO` y `UsuarioDAO`), nunca directamente desde las páginas.

El script completo para crear la base es `CompuTechDB.sql`; se puede ejecutar varias veces porque borra y vuelve a crear la base.

1. Objetivos

Objetivo general: explicar la estructura de la base de datos CompuTechDB y justificar, tabla por tabla, cómo respalda el funcionamiento de la aplicación web de gestión de productos.

Objetivos específicos:

- Describir las 8 tablas con sus columnas, tipos de datos, claves y restricciones.
- Mostrar las relaciones entre tablas y las reglas que garantizan la consistencia de los datos.
- Documentar las vistas y los procedimientos almacenados que realizan las operaciones CRUD (crear, leer, actualizar y eliminar).
- Presentar consultas de comprobación con sus resultados esperados.
- Explicar cómo generar el archivo SQL completo para llevar la base a otro equipo (portabilidad).

2. Ficha técnica del entorno

Elemento	Detalle
Motor de base de datos	Microsoft SQL Server, administrado con SQL Server Management Studio (SSMS)
Base de datos	CompuTechDB
Lenguaje de consulta	T-SQL
Contenido de la base	8 tablas, 4 vistas y 8 procedimientos almacenados
Archivo del script	`CompuTechDB.sql`
Aplicación	PaginaWeb-Proyecto (se empaqueta como `SistemaProductos.war`)
Lenguaje y servidor	Java 17, Tomcat 11 (Servlet 6.1, JSP 4.0, JSTL 3.0)
Gestión del proyecto	Maven (NetBeans)
Conexión	JDBC con `mssql-jdbc` 12.8.1
Cadena de conexión	`jdbc:sqlserver://localhost:1433;databaseName=CompuTechDB;encrypt=true;trustServerCertificate=true`

Elemento	Detalle
Credenciales de conexión	Archivo `db.properties` (por defecto usuario `sa`)

3. Estructura general de CompuTechDB

Las 8 tablas se agrupan en tres bloques: catálogo (Categoria, Marca, Proveedor, Producto), cuentas (Usuario) y compras (Carrito, Pedido, DetallePedido). **Producto** es la tabla central: casi todo se relaciona con ella.

N°	Tabla	Bloque	Función	Clave primaria
1	Categoria	Catálogo	Agrupa los productos (Laptops, Monitores, Audio…)	id_categoria
2	Marca	Catálogo	Lista de marcas (Lenovo, Logitech, ASUS…)	id_marca
3	Proveedor	Catálogo	Empresas que abastecen los productos	id_proveedor
4	Producto	Catálogo	Cada equipo o accesorio con precio, stock y estado	codigo
5	Usuario	Cuentas	Personas que inician sesión, con su rol	login
6	Carrito	Compras	Productos que un usuario va añadiendo para comprar	login + codigo
7	Pedido	Compras	Cabecera de cada compra confirmada	id_pedido
8	DetallePedido	Compras	Productos, cantidades y precios de cada pedido	id_detalle

Relaciones entre tablas

Cada relación es de uno a muchos: el lado "uno" es la tabla que se referencia y el lado "muchos" la que guarda la clave foránea.

- Categoria → Producto: una categoría tiene muchos productos.
- Marca → Producto: una marca tiene muchos productos (la marca es opcional).
- Proveedor → Producto: un proveedor abastece muchos productos (opcional).
- Usuario → Carrito: un usuario tiene muchos renglones en su carrito.
- Producto → Carrito: un producto puede estar en muchos carritos.
- Usuario → Pedido: un usuario hace muchos pedidos.
- Pedido → DetallePedido: un pedido tiene muchos detalles.
- Producto → DetallePedido: un producto aparece en muchos detalles de pedido.

Carrito y DetallePedido son tablas intermedias: resuelven la relación de muchos a muchos entre Usuario y Producto, y entre Pedido y Producto.

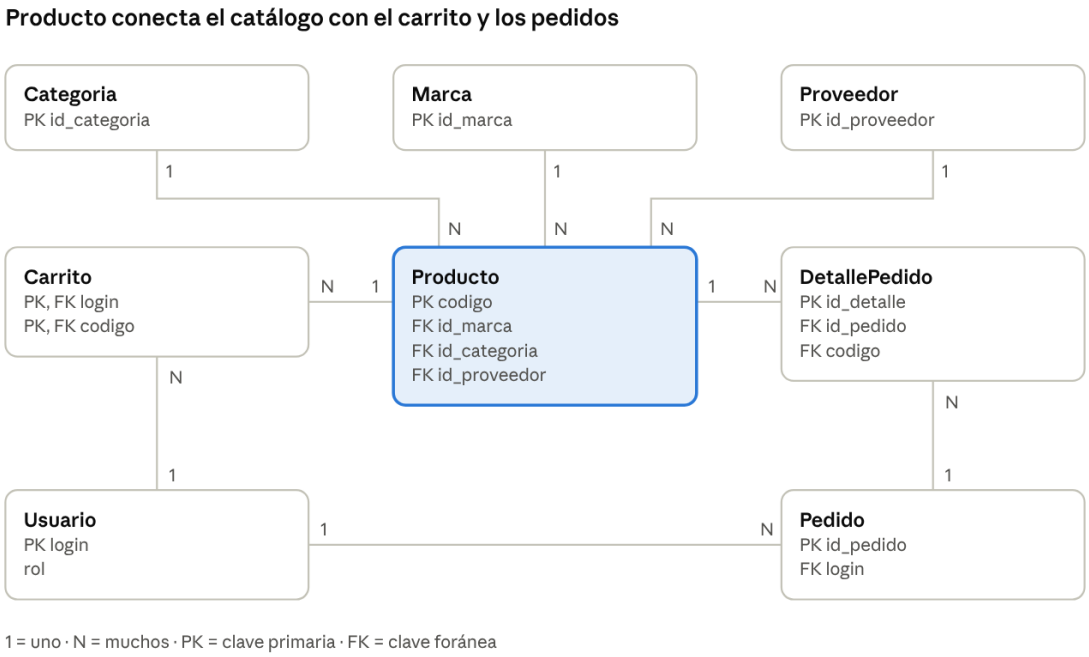


diagrama entidad-relación · 8 tablas, 8 relaciones

Producto está en el centro: recibe las claves de Categoria, Marca y Proveedor, y es el punto de unión con Carrito y DetallePedido. Usuario aparece dos veces porque se relaciona con el carrito y con los pedidos.

4. Diccionario de datos y evidencias

El diccionario de datos describe, para cada una de las 8 tablas, sus columnas, el tipo de dato, las restricciones y su uso en la aplicación. Cada tabla incluye su evidencia: el archivo de código que la consulta o modifica.

Criterio de presentación: se utiliza T-SQL sencillo y verificable, con instrucciones como USE, GO y SELECT. En este trabajo se consultan únicamente las tablas reales de CompuTechDB, y las operaciones de datos son CREATE, INSERT, UPDATE y DELETE (CRUD).

Tabla	Evidencia en el proyecto
Categoria	ProductoDAO: se une a Producto en todas las consultas y se consulta al insertar y actualizar
Marca	ProductoDAO: se une a Producto y se crea sola al guardar una marca nueva
Proveedor	Solo en el script SQL; la aplicación todavía no la usa
Producto	ProductoDAO (listar, buscar, insertar, actualizar, actualizarStock, eliminar) y ProductoService; PedidoDAO descuenta el stock al confirmar una compra
Usuario	UsuarioDAO (buscar, insertar) y UsuarioService (autenticar, registrar)
Carrito	CarritoDAO (listar, guardar, quitar, vaciar), llamado desde CarritoServlet

Tabla	Evidencia en el proyecto
Pedido	`PedidoDAO` (registrar, listar), llamado desde `PedidoServlet`
DetallePedido	`PedidoDAO` (registrar, listar), llamado desde `PedidoServlet`

4.1 Tablas del catálogo: Categoria, Marca y Proveedor

Estas tres tablas son listas de apoyo: guardan datos que se repiten en muchos productos para no escribirlos una y otra vez. Una categoría, marca o proveedor se escribe una sola vez y los productos la referencian por su número.

Tabla Categoria

Guarda las 8 categorías de la tienda: Laptops, Computadoras, Monitores, Periféricos, Componentes, Almacenamiento, Redes y Audio. El formulario de productos usa esta lista en el menú desplegable.

Columna	Tipo	Restricciones	Descripción
id_categoria	INT	Clave primaria, autonumérico	Número que identifica la categoría
nombre	NVARCHAR(50)	Obligatorio, único	Nombre de la categoría

Tabla Marca

Guarda las marcas (ASUS, Corsair, Kingston, Lenovo, Logitech, MSI, Redragon, Teros y Gamer Machine). Si el administrador escribe una marca nueva al registrar un producto, la aplicación la crea sola en esta tabla.

Columna	Tipo	Restricciones	Descripción
id_marca	INT	Clave primaria, autonumérico	Número que identifica la marca
nombre	NVARCHAR(60)	Obligatorio, único	Nombre de la marca

Tabla Proveedor

Guarda las empresas que abastecen los productos, con sus datos de contacto. Por ahora la aplicación no la muestra en pantalla; sirve como registro para el inventario.

Columna	Tipo	Restricciones	Descripción
id_proveedor	INT	Clave primaria, autonumérico	Número que identifica al proveedor
nombre	NVARCHAR(100)	Obligatorio	Razón social
ruc	VARCHAR(11)	Único, opcional	RUC de la empresa
telefono	VARCHAR(20)	Opcional	Teléfono de contacto
correo	NVARCHAR(120)	Opcional	Correo de contacto

4.2 Tabla Producto

Producto guarda cada equipo o accesorio que vende CompuTech: 12 productos iniciales (P001 a P012). Es la tabla que más usa la aplicación: el catálogo, el listado, el buscador, los reportes y el carrito leen de aquí.

Columna	Tipo	Restricciones	Descripción
codigo	VARCHAR(10)	Clave primaria	Código único del producto (P001, P002…); la app lo guarda en mayúsculas
nombre	NVARCHAR(120)	Obligatorio	Nombre comercial
id_marca	INT	Clave foránea a Marca, opcional	Marca del producto
id_categoria	INT	Clave foránea a Categoria, obligatorio	Categoría del producto
id_proveedor	INT	Clave foránea a Proveedor, opcional	Proveedor que lo abastece
precio	DECIMAL(10,2)	Obligatorio, mayor que 0	Precio en soles (S/)
stock	INT	Obligatorio, mayor o igual a 0	Unidades en inventario
estado	VARCHAR(20)	Solo 'Disponible' o 'Agotado'; por defecto 'Disponible'	Si se puede vender o no
descripcion	NVARCHAR(500)	Por defecto vacío	Características principales
imagen	VARCHAR(100)	Por defecto 'mouse.svg'	Ilustración de respaldo si no hay foto

Cómo funciona el estado

El estado se guarda tal como lo elige el administrador o el almacenero. La única regla automática es que **con stock 0 el estado siempre es Agotado**. Un producto marcado Agotado, aunque tenga stock, no se muestra en el reporte de disponibles, no se puede añadir al carrito y sus botones de compra se desactivan.

Fotos y la columna imagen

La foto real de cada producto es un archivo `img/productos/CODIGO.jpg`. La columna `imagen` solo guarda el nombre de una ilustración (por ejemplo `laptop.svg`) que se muestra cuando no existe esa foto.

4.3 Tabla Usuario y roles

Usuario guarda las cuentas que inician sesión. El rol de cada cuenta decide qué puede hacer en la aplicación. Las personas que se registran desde la página de registro siempre reciben el rol USUARIO.

Columna	Tipo	Restricciones	Descripción

Columna	Tipo	Restricciones	Descripción
login	VARCHAR(20)	Clave primaria	Nombre de usuario (de 4 a 20 caracteres)
clave_hash	VARCHAR(200)	Obligatorio	Contraseña cifrada, nunca en texto plano
nombre	NVARCHAR(100)	Obligatorio	Nombre completo
correo	NVARCHAR(120)	Obligatorio	Correo electrónico
rol	VARCHAR(10)	Solo 'ADMIN', 'ALMACENERO' o 'USUARIO'	Permisos de la cuenta

Roles

Rol	Quién es	Qué puede hacer
ADMIN	Administrador	Registrar, editar y eliminar productos; actualizar stock; ver todos los reportes
ALMACENERO	Personal de almacén	Ver el inventario, actualizar solo el stock y el estado, ver reportes
USUARIO	Cliente	Ver el catálogo, buscar productos, usar el carrito y comprar por WhatsApp

Cifrado de la contraseña

La contraseña se guarda con el algoritmo PBKDF2 (HMAC-SHA256, 65 536 vueltas) más una sal aleatoria de 16 bytes. El texto guardado tiene la forma `sal:hash`, ambos en Base64. Al iniciar sesión, la aplicación repite el cálculo con la sal guardada y compara el resultado; así nadie puede leer la contraseña original desde la base.

4.4 Tablas de compras: Carrito, Pedido y DetallePedido

Estas tres tablas forman el proceso de compra y la aplicación **ya las usa**: cada producto que el cliente añade se guarda en Carrito, y el botón "Confirmar compra" crea el pedido en Pedido y DetallePedido, descuenta el stock y vacía el carrito.

Tabla Carrito

Un renglón por cada producto que un usuario tiene en su carrito. La clave primaria es doble (login + codigo), así un mismo producto no se repite en el carrito de una persona. Si se borra el usuario o el producto, sus renglones se borran solos.

Columna	Tipo	Restricciones	Descripción
login	VARCHAR(20)	Clave primaria y foránea a Usuario	Dueño del carrito
codigo	VARCHAR(10)	Clave primaria y foránea a Producto	Producto añadido

Columna	Tipo	Restricciones	Descripción
cantidad	INT	Mayor que 0	Unidades que quiere comprar
fecha	DATETIME2	Por defecto la fecha y hora actual	Cuándo se añadió

Tabla Pedido

La cabecera de cada compra: quién compró, cuándo y por cuánto. El estado indica en qué etapa va el pedido.

Columna	Tipo	Restricciones	Descripción
id_pedido	INT	Clave primaria, autonumérico	Número de pedido
login	VARCHAR(20)	Clave foránea a Usuario	Quién hizo la compra
fecha	DATETIME2	Por defecto la fecha y hora actual	Cuándo se hizo
total	DECIMAL(12,2)	Mayor o igual a 0	Importe total en soles
estado	VARCHAR(20)	'Pendiente', 'Pagado', 'Enviado' o 'Cancelado'; por defecto 'Pendiente'	Etapa del pedido

Tabla DetallePedido

Los productos de cada pedido, uno por renglón. Guarda el **precio unitario del momento de la compra**, para que un cambio posterior de precio no altere pedidos antiguos. Si se borra un pedido, sus detalles se borran con él.

Columna	Tipo	Restricciones	Descripción
id_detalle	INT	Clave primaria, autonumérico	Número del detalle
id_pedido	INT	Clave foránea a Pedido	Pedido al que pertenece
codigo	VARCHAR(10)	Clave foránea a Producto	Producto comprado
cantidad	INT	Mayor que 0	Unidades compradas
precio_unitario	DECIMAL(10,2)	Mayor que 0	Precio por unidad al comprar

Un mismo producto no puede repetirse dentro de un pedido (restricción única sobre id_pedido + codigo).

5. Relaciones, restricciones y consistencia

La base de datos se protege a sí misma con claves foráneas y restricciones CHECK, de modo que no se puedan guardar datos incoherentes aunque se inserten desde fuera de la aplicación.

Regla	Cómo se cumple
No hay códigos de producto repetidos	Clave primaria en Producto.codigo

Regla	Cómo se cumple
Nombres de categoría y marca no se repiten	Restricción UNIQUE en Categoria.nombre y Marca.nombre
El precio es mayor que cero	CHECK en Producto.precio
El stock nunca es negativo	CHECK en Producto.stock
El estado solo puede ser Disponible o Agotado	CHECK en Producto.estado
Con stock 0 el producto queda Agotado	Regla de la aplicación (ProductoService) al guardar
El rol solo puede ser ADMIN, ALMACENERO o USUARIO	CHECK en Usuario.rol
Un producto necesita una categoría que exista	Clave foránea Producto → Categoria
Un pedido pertenece a un usuario real	Clave foránea Pedido → Usuario
No se borra un producto que ya fue vendido	Clave foránea DetallePedido → Producto sin borrado en cascada
El carrito se limpia al borrar usuario o producto	Borrado en cascada en Carrito
Los detalles se borran con su pedido	Borrado en cascada en DetallePedido → Pedido
El RUC de un proveedor no se repite	UNIQUE en Proveedor.ruc

También hay dos índices (`IX_Producto_categoria` e `IX_Producto_marca`) que aceleran los filtros del catálogo por categoría y marca.

Claves foráneas y qué pasa al borrar

Cada clave foránea define qué ocurre al borrar el registro padre. Donde no hay borrado en cascada, SQL Server rechaza la eliminación para no dejar registros huérfanos.

Clave foránea (tabla hija)	Tabla padre	Al borrar el registro padre
Producto.id_categoria	Categoria	Se rechaza si hay productos en esa categoría
Producto.id_marca	Marca	Se rechaza si hay productos de esa marca
Producto.id_proveedor	Proveedor	Se rechaza si hay productos de ese proveedor
Carrito.login	Usuario	Se borran sus renglones del carrito (cascada)
Carrito.codigo	Producto	Se borra el producto de los carritos (cascada)
Pedido.login	Usuario	Se rechaza si el usuario tiene pedidos
DetallePedido.id_pedido	Pedido	Se borran sus detalles (cascada)
DetallePedido.codigo	Producto	Se rechaza si el producto ya fue vendido

La última fila tiene una consecuencia práctica: como la aplicación ya guarda pedidos, eliminar un producto ya vendido falla y conviene mostrar un mensaje claro al administrador (hoy `ProductoDAO.eliminar` lanza un error genérico de base de datos).

6. Vistas de la base de datos

Una vista es una consulta guardada con nombre que se usa como si fuera una tabla. CompuTechDB tiene 4 vistas que reúnen datos de varias tablas, para no repetir los mismos JOIN en cada consulta.

Vista	Qué muestra	Tablas que une	Equivale a
vw_ProductoDetalle	Cada producto con el nombre de su marca, categoría y proveedor, y el valor del inventario (precio × stock)	Producto, Categoria, Marca, Proveedor	El catálogo y el listado
vw_ProductosDisponibles	Productos con estado Disponible y stock mayor que cero	vw_ProductoDetalle	Reporte "Productos disponibles"
vw_ResumenInventario	Una fila con total de productos, disponibles, agotados, stock total y valor del inventario	Producto	Reporte "Resumen de productos"
vw_PedidoDetalle	Cada producto de cada pedido con el cliente y el subtotal	Pedido, Usuario, DetallePedido, Producto	Página Pedidos (historial de compras)

Definición de la vista principal:

```
CREATE VIEW dbo.vw_ProductoDetalle AS
SELECT p.codigo, p.nombre,
      COALESCE(m.nombre, N'')  AS marca,
      c.nombre                 AS categoria,
      COALESCE(pr.nombre, N'') AS proveedor,
      p.precio, p.stock, p.estado, p.descripcion, p.imagen,
      p.precio * p.stock       AS valor_inventario
FROM dbo.Producto p
INNER JOIN dbo.Categoria c  ON c.id_categoria  = p.id_categoria
LEFT  JOIN dbo.Marca m      ON m.id_marca      = p.id_marca
LEFT  JOIN dbo.Proveedor pr ON pr.id_proveedor = p.id_proveedor;
```

Se usa INNER JOIN con Categoria porque todo producto debe tener categoría, y LEFT JOIN con Marca y Proveedor porque son opcionales: así un producto sin marca igual aparece en el listado.

7. Procedimientos almacenados

Un procedimiento almacenado es un bloque de T-SQL guardado en la base de datos que se ejecuta con `EXEC`. CompuTechDB tiene 8 procedimientos que repiten las operaciones que hoy hace la aplicación con `ProductoDAO` y `UsuarioDAO`, pero dentro del servidor. La aplicación Java sigue usando sentencias SQL con `PreparedStatement`; los procedimientos sirven para administrar la base desde SSMS y para demostrar el CRUD.

Procedimiento	CRUD	Qué hace	Equivalente en Java
sp_Producto_Insertar	Create	Registra un producto; crea la marca si no existe	ProductoDAO.insertar
sp_Producto_Listar	Read	Lista todos los productos	ProductoDAO.listar
sp_Producto_Buscar	Read	Busca por código, nombre, categoría o todos	ProductoDAO.buscar
sp_Producto_Actualizar	Update	Modifica los datos de un producto	ProductoDAO.actualizar
sp_Producto_ActualizarStock	Update	Cambia solo el stock y el estado	ProductoDAO.actualizarStock
sp_Producto_Eliminar	Delete	Elimina un producto	ProductoDAO.eliminar
sp_Usuario_Registrar	Create	Crea una cuenta siempre con rol USUARIO	UsuarioDAO.insertar
sp_Usuario_Buscar	Read	Busca una cuenta por su login	UsuarioDAO.buscar

Todos los procedimientos de producto aplican la misma regla que la aplicación: **con stock 0 el estado queda Agotado**. Los que modifican o borran lanzan un error (THROW) si el código no existe, y los que escriben validan que la categoría exista.

7.1 Operaciones CRUD implementadas en CompuTechDB

Secuencia de prueba que recorre las cuatro operaciones sobre un producto nuevo (P013):

```
USE CompuTechDB;
GO
-- C: crear
EXEC dbo.sp_Producto_Insertar @codigo = 'P013', @nombre = N'Mouse Inalámbrico M170',
    @marca = N'Logitech', @categoria = N'Periféricos', @precio = 49.90, @stock = 20;
-- R: leer
EXEC dbo.sp_Producto_Buscar @criterio = 'codigo', @texto = N'P013';
-- U: actualizar (stock 0 => estado Agotado)
EXEC dbo.sp_Producto_ActualizarStock @codigo = 'P013', @stock = 0;
EXEC dbo.sp_Producto_Buscar @criterio = 'codigo', @texto = N'P013';
-- D: eliminar
EXEC dbo.sp_Producto_Eliminar @codigo = 'P013';
GO
```

Resultado esperado: la primera búsqueda muestra P013 con stock 20 y estado Disponible; la segunda, con stock 0 y estado Agotado; después de eliminar, el inventario vuelve a tener 12 productos.

8. Consultas y resultados de comprobación

Estas consultas comprueban que la base se creó completa. Los resultados indicados se calcularon a partir de los datos iniciales del script; al ejecutarlas en SSMS sobre una base recién creada deben coincidir.

```
USE CompuTechDB;
GO
-- 1. Tablas creadas
SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_TYPE = 'BASE
```

```
TABLE' ORDER BY TABLE_NAME;
-- 2. Resumen del inventario
SELECT * FROM dbo.vw_ResumenInventario;
-- 3. Productos y unidades por categoría
SELECT categoria, COUNT(*) AS productos, SUM(stock) AS unidades
FROM dbo.vw_ProductoDetalle GROUP BY categoria ORDER BY categoria;
-- 4. Productos agotados
SELECT codigo, nombre, stock, estado FROM dbo.Producto WHERE estado = 'Agotado';
-- 5. Productos disponibles
SELECT COUNT(*) AS disponibles FROM dbo.vw_ProductosDisponibles;
-- 6. Cuentas y roles
SELECT login, rol FROM dbo.Usuario;
```

Consulta	Resultado esperado
1. Tablas creadas	8 filas: Carrito, Categoria, DetallePedido, Marca, Pedido, Producto, Proveedor, Usuario
2. Resumen del inventario	12 productos, 11 disponibles, 1 agotado, 157 unidades de stock, valor del inventario S/ 101 173.00
3. Por categoría	Almacenamiento 1 producto (30 unidades); Audio 1 (14); Componentes 2 (24); Computadoras 1 (3); Laptops 2 (12); Monitores 1 (9); Periféricos 3 (59); Redes 1 (6)
4. Agotados	P012, Laptop Lenovo IdeaPad 3 15" Ryzen 5, stock 0, estado Agotado
5. Disponibles	11
6. Cuentas	admin (ADMIN) y almacen (ALMACENERO)

La consulta 2 es la que debe coincidir con el reporte "Resumen de productos" de la aplicación: ambos cuentan como agotado todo producto que no esté Disponible con stock mayor que cero.

Ver en SSMS los datos guardados desde la página

Todo lo que se hace en la página queda guardado en la base. Para verlo, abrir SSMS, conectarse al mismo servidor que usa db.properties, elegir la base CompuTechDB y ejecutar la consulta de la tabla correspondiente. En cada fila de la tabla siguiente, la acción de la izquierda agrega o cambia filas en la tabla de la derecha.

Acción en la página	Tabla donde se guarda	Consulta para verla
Registrarse como cliente	Usuario	`SELECT login, nombre, correo, rol FROM dbo.Usuario;`
Registrar o editar un producto (administrador)	Producto y, si la marca es nueva, Marca	`SELECT * FROM dbo.vw_ProductoDetalle;`
Cambiar stock o estado (almacenero o administrador)	Producto	`SELECT codigo, nombre, stock, estado FROM dbo.Producto;`
Añadir un producto al carrito	Carrito	`SELECT * FROM dbo.Carrito;`

Acción en la página	Tabla donde se guarda	Consulta para verla
Confirmar la compra	Pedido y DetallePedido; baja el stock y se vacía el Carrito	`SELECT * FROM dbo.vw_PedidoDetalle ORDER BY id_pedido DESC;`

La contraseña de cada usuario se ve en la columna `clave_hash` como texto cifrado, nunca como la contraseña real. El botón "Pedir por WhatsApp" abre WhatsApp y no guarda nada en la base: solo "Confirmar compra" crea el pedido.

Al confirmar una compra, la aplicación hace todo en una sola transacción: si algún producto no tiene stock suficiente, no se guarda nada y el stock no cambia.

9. Procedimiento real para generar el archivo SQL completo

El archivo SQL es la forma más portátil de llevar la base: es un solo archivo de texto que funciona en cualquier equipo con SQL Server, sin copiar archivos internos del motor. `CompuTechDB.sql` ya contiene la estructura completa, las vistas, los procedimientos y los datos iniciales. Si después se agregan datos desde la aplicación y se quieren llevar también, hay que generar el script otra vez desde SSMS.

9.1 Generar el script desde SSMS

1. En SSMS, clic derecho sobre la base `CompuTechDB` → Tareas → Generar scripts.
2. Elegir "Generar scripts de toda la base de datos y de todos los objetos de base de datos".
3. En Opciones de scripting, guardar en un solo archivo (Save as script file, Single script file) con codificación Unicode, para no perder tildes ni la ñ.
4. En Avanzadas, poner "Tipos de datos de los que se generará el script" en **Esquema y datos** (Schema and data) y "Comprobar si existe el objeto" en True.
5. En "Generar scripts para la versión de servidor", elegir la versión de SQL Server más antigua que tendrá el otro equipo.
6. Finalizar y guardar el archivo `.sql`.

El script generado así no incluye la sentencia `CREATE DATABASE`: en el otro equipo hay que crear antes una base vacía llamada `CompuTechDB` y ejecutar el script sobre ella. El archivo `CompuTechDB.sql` del proyecto sí la incluye, y por eso basta con ejecutarlo.

9.2 Ejecutar el script en otro equipo

Los nombres de las opciones de SSMS pueden variar según la versión y el idioma. Al terminar la ejecución, la base queda así.

Al ejecutar `CompuTechDB.sql` la base queda con datos de ejemplo listos para probar la aplicación.

Tabla	Datos iniciales
Categoria	8 categorías
Marca	9 marcas
Proveedor	4 proveedores

Tabla	Datos iniciales
Producto	12 productos (P001 a P012); P012 está Agotado con stock 0
Usuario	2 cuentas: admin (ADMIN) y almacen (ALMACENERO)
Carrito, Pedido, DetallePedido	Vacías

Los nombres, precios y proveedores de los productos son datos de ejemplo; se pueden reemplazar por los reales. Las cuentas iniciales son `admin` / `admin123` y `almacen` / `almacen123`; conviene cambiar las contraseñas después de la primera prueba.

Pasos para crear la base

1. Abrir SQL Server Management Studio (SSMS) y conectarse con una cuenta con permisos, por ejemplo `sa`.
2. Abrir el archivo `CompuTechDB.sql` y ejecutarlo completo (F5). Borra y recrea la base `CompuTechDB`.
3. Revisar el resultado final: la primera consulta debe listar las 8 tablas y la segunda los 12 productos.
4. Comprobar que `db.properties` tenga el usuario y la contraseña de SQL Server (por defecto `sa` / `123`).
5. Desplegar el proyecto en Tomcat e iniciar sesión con `admin`.

9.3 Otras formas de hacer la base portátil

Método	Qué se lleva	Cuándo conviene
Script `.sql`	Estructura, vistas, procedimientos y datos como texto	Opción recomendada: funciona entre versiones de SQL Server y se puede abrir y revisar
Copia de seguridad `.bak`	La base completa tal como está hoy	Para llevar los datos actuales; el otro equipo debe tener la misma versión de SQL Server o una más nueva
Paquete `.bacpac`	Estructura y datos en un solo archivo	Para pasar la base entre equipos o a un servidor en la nube
Copiar archivos `.mdf` y `.ldf`	Los archivos internos de la base	No recomendado: hay que separar la base del servidor antes de copiar y es fácil dañarla

Para la copia de seguridad, el comando es:

```
BACKUP DATABASE CompuTechDB TO DISK = N'C:\Respaldo\CompuTechDB.bak' WITH INIT, FORMAT;
```

Y para restaurarla en el otro equipo: clic derecho en Bases de datos → Restaurar base de datos → Dispositivo, y elegir el archivo `.bak`. El paquete `.bacpac` se genera desde SSMS con Tareas → Exportar aplicación de capa de datos, y se abre con Importar aplicación de capa de datos.

9.4 Configurar la aplicación en el otro equipo

La aplicación lee la conexión de src/main/resources/db.properties, así que solo hay que editar ese archivo y volver a desplegar. Si SQL Server está instalado como instancia con nombre (por ejemplo SQL Server Express), el servidor se escribe con la instancia y sin puerto:

```
db.host=localhost\SQLEXPRESS
db.name=CompuTechDB
db.user=sa
db.password=su_clave
```

El proyecto incluye la carpeta bd con instalar_bd.bat (crea la base con sqlcmd), respaldar_bd.bat (genera el .bak) y un LEEME.txt con estos pasos.

10. Validación del SQL obtenido

El script se revisó de forma automática en su estructura, pero **todavía no se ejecutó en un SQL Server real** durante la redacción de este informe. La ejecución completa es el último paso de validación y está en la lista de comprobación de abajo. Los servlets (CarritoServlet, PedidoServlet, ProductoServlet) y las páginas JSP tampoco se compilaron aquí, porque necesitan las librerías de Tomcat: hay que compilar el proyecto con Maven en NetBeans.

Revisión	Resultado
Lotes del script separados por GO	31 lotes
Paréntesis y bloques BEGIN/END balanceados en cada lote	Sin problemas
CREATE VIEW y CREATE PROCEDURE son la primera sentencia de su lote (regla de T-SQL)	Cumplido
Objetos creados	8 tablas, 4 vistas, 8 procedimientos
Datos iniciales leídos del script	12 productos, con 11 disponibles y 1 agotado
Clases Java Producto, ProductoDAO, ProductoService y UsuarioDAO	Compilan sin errores

Lista de comprobación al ejecutar en SSMS

- ☐ Ejecutar CompuTechDB.sql completo (F5): la pestaña Mensajes no debe mostrar errores en rojo.
- ☐ La primera consulta final lista 8 tablas.
- ☐ La consulta de resumen muestra 12 productos, 11 disponibles, 1 agotado y stock total 157.
- ☐ Las consultas de vistas y procedimientos devuelven filas.
- ☐ Iniciar sesión en la aplicación con admin, cambiar un producto a Agotado con stock mayor que cero y comprobar que sigue Agotado al volver a abrirlo.
- ☐ Con un cliente, añadir un producto al carrito y confirmar la compra; luego ejecutar SELECT * FROM dbo.vw_PedidoDetalle y comprobar que aparece el pedido y que el stock bajó.

11. Resultados y conclusiones

- La base CompuTechDB tiene 8 tablas relacionadas por 8 claves foráneas; **Producto** es la tabla central y concentra la información que muestra la aplicación.
- Los datos que se repiten (categorías, marcas, proveedores) están en tablas propias, lo que evita escribirlos varias veces y errores de ortografía entre productos.
- Las restricciones CHECK, UNIQUE y las claves foráneas hacen que la propia base rechace precios no válidos, stock negativo, estados o roles inexistentes y códigos repetidos.
- El problema original (un producto marcado Agotado se guardaba como Disponible) se corrigió en `ProductoService`: ahora se respeta el estado elegido y solo se fuerza Agotado cuando el stock es 0.
- Las 4 vistas y los 8 procedimientos cubren las operaciones CRUD y los dos reportes de la aplicación.
- Carrito, Pedido y DetallePedido están conectadas a la aplicación: el carrito y las compras confirmadas se guardan en la base y se pueden ver con SELECT en SSMS.
- La base es portable porque todo su contenido está en un solo archivo `.sql` y la conexión se configura en `db.properties`.

12. Recomendaciones para uso académico

- Ejecutar el script en SSMS y capturar pantallas de las tablas, vistas y resultados como evidencia del trabajo.
- Cambiar las contraseñas iniciales (`admin123` y `almacen123`) y no usar la cuenta `sa` ni la clave del archivo `db.properties` fuera de un entorno de práctica.
- Antes de modificar la base, hacer una copia de seguridad `.bak` y guardar el script en un repositorio para llevar el historial de cambios.
- Los nombres, precios y proveedores de los productos son datos de ejemplo; reemplazarlos por los reales del trabajo antes de presentarlo.
- Como mejora, agregar una restricción que impida un producto Disponible con stock 0 directamente en la base: `CHECK (stock > 0 OR estado = 'Agotado')`. Hoy esa regla la cumple solo la aplicación.
- Hacer una compra de prueba (añadir al carrito y confirmar) y capturar las filas nuevas de Carrito, Pedido y DetallePedido como evidencia.
- Revisar la cantidad de vueltas del cifrado PBKDF2 (hoy 65 536) con las recomendaciones de seguridad vigentes.

Referencias

- Archivos del proyecto: `CompuTechDB.sql`, `ProductoDAO.java`, `UsuarioDAO.java`, `ProductoService.java`, `UsuarioService.java`, `db.properties` y `pom.xml`.
- Codd, E. F. (1970). A Relational Model of Data for Large Shared Data Banks. *Communications of the ACM*, 13(6), 377–387.
- Moriarty, K., Kaliski, B. y Rusch, A. (2017). PKCS #5: Password-Based Cryptography Specification Version 2.1. RFC 8018.
- Microsoft. Documentación de SQL Server y Transact-SQL (CREATE TABLE, CREATE VIEW, CREATE PROCEDURE, BACKUP DATABASE), Microsoft Learn.